



TERM FINANCE

Term Finance Security Assessment Report

Version: 2.0

August, 2026

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	3
Findings Summary	3
Detailed Findings	5
Summary of Findings	6
Swap Output Recipient Is Not Enforced In The Aggregator Branch	7
IntentFilled Reports A Servicing Fee In The Borrow Fee Field	9
KyberSwapFacet Does Not Enforce The Minimum Output Or The Recipient	10
Presigned Order Events Do Not Index Their Parameters	11
Public swap() No Longer Enforces A Minimum Output	12
Miscellaneous General Comments	13
A Vulnerability Severity Classification	14

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Term Finance components in scope. The review focused solely on the security aspects of the Solidity contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Term Finance components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Term Finance components in scope.

Overview

This assessment covers an update to the previously reviewed Term Finance codebase, focused on changes to the existing intent, strategy, swap, and term initialisation flows.

The update extends observability and control. `TermStrategyFacet` now emits `IntentFilled` for synthetic strategy-backed fills, the intent facets emit presigned-order lifecycle events, and facet versions are exposed onchain through `Versionable`.

The release also tightens execution paths: strategy validation now uses `isTermApproved()`, `SwapRouterFacet` hardens direct swaps and receiver handling, `KyberSwapFacet` adds a Kyber hook, `TermInitializer` adds auctionless setup, and minting changes add zero-amount guards and `redemptionTimestamp` fee proration.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [Term Finance](#) repository.

The scope of this time-boxed review was strictly limited to Solidity file changes under `contracts/` introduced in commit [151426a..3aa99f9](#), including the logic changes in PRs #1666, #1682, #1683, #1685, #1686, #1688, #1754, #1761, #1771, #1777, #1778, #1795, and #1796.

The development team's responses to the identified issues were assessed at commit [3aa99f9](#), that is, the same commit as the original review. No code changes were made in response to the findings.

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

For the Solidity components, the manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 6 issues during this assessment. Categorised by their severity:

- Low: 1 issue.
- Informational: 5 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Term Finance components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open**: the finding has not been addressed by the project team.
- **Resolved**: the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed**: the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
TRF7-01	Swap Output Recipient Is Not Enforced In The Aggregator Branch	Low	Closed
TRF7-02	IntentFilled Reports A Servicing Fee In The Borrow Fee Field	Informational	Closed
TRF7-03	KyberSwapFacet Does Not Enforce The Minimum Output Or The Recipient	Informational	Closed
TRF7-04	Presigned Order Events Do Not Index Their Parameters	Informational	Closed
TRF7-05	Public swap() No Longer Enforces A Minimum Output	Informational	Closed
TRF7-06	Miscellaneous General Comments	Informational	Resolved

TRF7-01	Swap Output Recipient Is Not Enforced In The Aggregator Branch		
Assets	contracts/facets/SwapRouterFacet.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Swaps routed through the external aggregator ignore the requested output recipient, so the swap proceeds even when the output is delivered to an address other than the one the entry point asked for.

The `swap()` and `swapHook()` entry points pass a `receiver` argument into `_swapInternal()` so the swap output goes to a fixed address. The Pendle branches use this argument. The branch that routes through the external aggregator does not:

```
contracts/facets/SwapRouterFacet.sol::_swapInternal()
function _swapInternal(address tokenIn, uint256 amountIn, address receiver, SwapRouterData memory data) internal {
    // ... snip ...
} else {
    ERC20(tokenIn).safeTransfer(pendleSwap, amountIn);
    SwapData memory swapData = abi.decode(data.swapData, (SwapData));
    IPSwapAggregator(pendleSwap).swap(tokenIn, amountIn, swapData); // @audit receiver is not applied here
}
}
```

The output recipient for this branch comes from the aggregator calldata inside `data.swapData`, which the caller controls. The facet does not confirm that the output reaches `receiver`, and it does not check the output balance after the swap.

This behaviour cannot be used to take another party's funds in the current design. `swapHook()` runs behind `onlyFlashLoanContext`, which only permits the address that started the flash loan, and `TermFlashLoanExecutorFacet` measures the actual output token balance of the diamond after each action. A swap that sends its output elsewhere produces a measured output of zero, the flash loan cannot be repaid, and the transaction reverts.

This issue has a low impact as the exposure is limited to the caller acting against funds the caller already controls, and the downstream balance check in `TermFlashLoanExecutorFacet` reverts any transaction where the output is redirected. This issue has a low likelihood as it requires a caller to supply aggregator calldata that redirects their own swap output, which offers the caller no gain.

Recommendations

Apply `receiver` in the aggregator branch where the aggregator supports a recipient parameter. Where it does not, record the output token balance before and after the swap and confirm the increase reaches `receiver`. This keeps the guarantee inside the facet instead of relying on `TermFlashLoanExecutorFacet` to reject a redirected swap.

Resolution

The development team has closed the issue with the following comment:

The Pendle Swap contract sends arbitrary calldata bytes to a router address ([0xBC17404b7bb500051c75C83E4aA5aE447D967811](#)), so there is no standard ABI that can be used to decode the entire calldata and replace the recipient address as we do for the other swap routes. As a result, we cannot apply the same recipient-address handling to this route.

We also decided not to implement the balance checks directly in `swapHook()`, since these checks are already performed at the executor level. Additionally, `swapHook()` can only be executed within a flash executor context, so the executor-level checks provide the necessary validation for this flow.

TRF7-02	IntentFilled Reports A Servicing Fee In The Borrow Fee Field	
Assets	contracts/facets/TermStrategyFacet.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The `IntentFilled` event emitted by the strategy sell flow reports a servicing fee in a field that offchain consumers read as a borrow fee, so indexed fee data for these events is mislabelled.

[PR #1666](#) added an `IntentFilled` event to `mintAndSellRepoToken()`. The value passed into the event as `proratedBorrowFee` is derived from the servicing fee, not a borrow fee:

```
contracts/facets/TermStrategyFacet.sol::mintAndSellRepoToken()
```

```
uint256 proratedFee = mul_ScalarTruncate(
    proRatedServicingFee,
    amountOfRepoTokensToMint
);
// ... snip ...
_emitIntentFilled(
    // ... snip ...
    proratedFee, // @audit servicing fee passed as proratedBorrowFee
    // ... snip ...
);
```

For real loan intents the `proratedBorrowFee` field carries a borrow fee. For this strategy sell flow the same field carries a servicing fee. Any offchain consumer that reads `IntentFilled.proratedBorrowFee`, such as the subgraph, will record a servicing fee as a borrow fee for these events.

This issue is rated as informational as there is no onchain impact. The mismatch only affects the accuracy of offchain accounting and reporting built on top of these events.

Recommendations

Consider confirming that the subgraph expects a servicing fee in this field for the strategy sell flow. If it does not, use a separate field or event for this case, or update the offchain indexing so the value is labelled correctly.

Resolution

The development team has closed the issue with the following comment:

The servicing fee charged during the mint is the only fee deducted as part of this order fill. The Term subgraph is designed to pick up this fee for accounting purposes.

TRF7-03	KyberSwapFacet Does Not Enforce The Minimum Output Or The Recipient	
Assets	contracts/facets/KyberSwapFacet.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

`kyberSwapHook()` forwards caller supplied calldata directly to the Kyber router without setting or checking the minimum return or the output recipient, so neither guarantee is held by the facet itself.

```
contracts/facets/KyberSwapFacet.sol::_kyberSwapInternal()
function _kyberSwapInternal(address tokenIn, uint256 amountIn, bytes memory swapCalldata) internal {
    ERC20(tokenIn).forceApprove(kyberMetaAggregationRouterV2, amountIn);
    kyberMetaAggregationRouterV2.functionCall(swapCalldata); // @audit minReturnAmount and dstReceiver live inside swapCalldata
    ERC20(tokenIn).forceApprove(kyberMetaAggregationRouterV2, 0);
}
```

Both the minimum return (`minReturnAmount`) and the output recipient (`dstReceiver`) are encoded inside `swapCalldata`. The facet does not pin the recipient to the diamond and does not read the output balance after the swap. `kyberSwapHook()` is also passed an `input.minOutputAmount`, but that value is only used in `previewKyberSwap()` and is never enforced during the swap.

This is the same pattern as the aggregator branch in `SwapRouterFacet`, and it is contained the same way. The hook runs only inside a flash loan context for the initiating address, and `TermFlashLoanExecutorFacet` reverts when the diamond does not receive the expected output.

This issue is rated as informational as the exposure is limited to the caller's own funds, and the downstream balance check in `TermFlashLoanExecutorFacet` reverts any swap that does not deliver the expected output to the diamond.

Recommendations

Consider reading the output token balance after the swap and confirming the increase is at least `input.minOutputAmount`. This would enforce both the minimum output and delivery to the diamond inside the facet, instead of trusting the contents of `swapCalldata`.

Resolution

The development team has closed the issue with the following comment:

`minOutputAmount` is checked at the `TermFlashLoanExecutorFacet` level. `kyberSwapHook()` can only be executed within the flash executor's context, so the check is enforced at the executor level rather than within the hook itself. This is consistent with the pattern used across our other action hooks, where validation that applies to the overall flash execution is performed at the executor level rather than duplicated within each individual hook.

TRF7-04	Presigned Order Events Do Not Index Their Parameters	
Assets	contracts/interfaces/ITermIntentEvents.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

None of the parameters on the new presigned order events are marked `indexed`, so offchain consumers cannot filter these logs by order hash and must scan and decode every event instead.

[PR #1778](#) added four events for presigned orders:

```
contracts/interfaces/ITermIntentEvents.sol
event PresignedLimitOrderSet(bytes32 orderHash, address signer);
event PresignedLimitOrderRevoked(bytes32 orderHash);
event PresignedSwapOrderSet(bytes32 orderHash, address signer);
event PresignedSwapOrderRevoked(bytes32 orderHash);
```

Without an indexed `orderHash`, topic filters cannot be used to retrieve the lifecycle events for a single order.

This issue is rated as informational as it is an efficiency point for offchain tooling and has no onchain impact.

Recommendations

Consider marking `orderHash` as `indexed` in all four events, and `signer` in the two set events if filtering by signer is useful. Note that changing the event signatures requires the offchain indexers to be updated to match.

Resolution

The development team has closed the issue with the following comment:

This suggestion makes sense, but none of our existing intent events ([ITermIntentEvents.sol](#)) index on `orderHash`. Since we already have live contracts using these existing events, we will keep the `indexed` decorator out for consistency. Our subgraph also indexes by `orderHash` without issue, so adding the decorator is not necessary for our current implementation.

TRF7-05	Public <code>swap()</code> No Longer Enforces A Minimum Output	
Assets	contracts/facets/SwapRouterFacet.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

The public `swap()` function does not check how much output the caller receives, so a caller who supplies weak swap calldata has no slippage protection at the facet level.

Prior to [PR #1685](#), `swap()` took a `minTokenOut` parameter and compared the caller's output token balance before and after the swap:

```
contracts/facets/SwapRouterFacet.sol::swap()
uint256 userBalanceBefore = ERC20(tokenOut).balanceOf(msg.sender);
_swapInternal(tokenIn, amountIn, data);
uint256 userBalanceAfter = ERC20(tokenOut).balanceOf(msg.sender);
if (userBalanceAfter - userBalanceBefore < minTokenOut) {
    revert InsufficientOutputAmount();
}
```

That pull request removed these parameters, the balance check, and the `InsufficientOutputAmount` error. `swap()` now performs no slippage check of its own, and protection depends entirely on the minimum encoded inside `data.swapData`. For Pendle swaps this is acceptable, because the Pendle router enforces its own minimum output. For the external aggregator branch there is no equivalent guarantee at the facet level.

This issue is rated as informational as the function moves the caller's own tokens using calldata the caller supplies, so a missing or weak minimum only affects that caller. There is no impact on other users or on protocol funds.

Recommendations

Consider restoring a minimum output check in `swap()`, or documenting that slippage protection for this function now depends on the minimum set inside `data.swapData` and is not enforced by the facet.

Resolution

The development team has closed the issue with the following comment:

PR #1685 reverts a change introduced in PR #1683. The changes from PR #1683 were never deployed onchain. Before deploying the changes, we decided to forgo the `minOutputAmount` check, as this was not a feature included in our original design. Therefore, PR #1685 is not removing or disabling an existing `minOutputAmount` check from deployed contracts; rather, it reverts an undeployed change before it was introduced into production.

TRF7-06	Miscellaneous General Comments	
Assets	contracts/*	
Status	Resolved: See Resolution	
Rating	Informational	

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Onchain Events Should Match Offchain Tracking Needs

Related Asset(s): `contracts/facets/TermStrategyFacet.sol`, `contracts/facets/TermControllerFacet.sol`, `contracts/interfaces/ITermIntentEvents.sol`

This update introduces new event surfaces for strategy-backed fills, presigned-order lifecycle actions, and EIP-712 domain updates.

As event coverage expands, the development team should ensure emitted events precisely match the information that offchain systems are expected to track. Where a top-level action implies meaningful subordinate actions or state transitions, relying on offchain consumers to reconstruct those details from context can make indexing and reconciliation harder than necessary.

Consider reviewing whether each operationally significant action exposes the full set of fields and event granularity required by downstream monitoring, indexing, and reporting systems.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team has acknowledged these findings, addressing them where appropriate.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'